

Singleton Design Pattern Real-Time Example in C# – Exception Logging

In this article, I am going to discuss the **Singleton Design Pattern Real-Time Example Exception Logging** in C# using ASP.NET MVC Application. Please read our previous article, discussing the [Singleton vs. Static Class in C#](#) with Examples. As part of this article, we will discuss How to Create a Custom Logger Library using the Singleton Design Pattern, which logs the Exceptions to an External File using the ASP.NET MVC application.

Singleton Design Pattern Real-Time Example Exception Logging using ASP.NET MVC Application:

Let us see the step-by-step process of how to implement the Singleton Design Pattern Real-Time Example Exception Logging using C# and ASP.NET MVC Application.

Step 1: Creating the Database

In this demo, we are going to use the following Employee table.

Id	Name	Gender	Salary	Department
1	Pranaya	Male	12345	IT
2	Anurag	Male	100000	Sales
3	Priyanka	Female	20000	IT
4	Preety	Female	30000	HR
5	James	Male	250000	HR
6	Pam	Female	30000	IT
7	Smith	Male	220000	Sales

So, please use the below SQL script to create the database EmployeeDB and the table Employee with the required test data.

```
-- Create a database Called EmployeeDB
CREATE DATABASE EmployeeDB
GO
USE EmployeeDB
GO
-- Create a table called Employee
CREATE TABLE Employee (
    [Id] INT IDENTITY (1, 1) NOT NULL,
    [Name] VARCHAR (50) NOT NULL,
    [Gender] VARCHAR (50) NOT NULL,
    [Salary] BIGINT NOT NULL,
    [Department] VARCHAR (50) NOT NULL,
    PRIMARY KEY CLUSTERED ([Id] ASC)
);
GO
-- Insert some test data to the Employee table
INSERT INTO Employee VALUES('Pranaya', 'Male', 12345, 'IT')
INSERT INTO Employee VALUES('Anurag', 'Male', 100000, 'Sales')
INSERT INTO Employee VALUES('Priyanka', 'Female', 20000, 'IT')
INSERT INTO Employee VALUES('Preety', 'Female', 30000, 'HR')
INSERT INTO Employee VALUES('James', 'Male', 250000, 'HR')
```

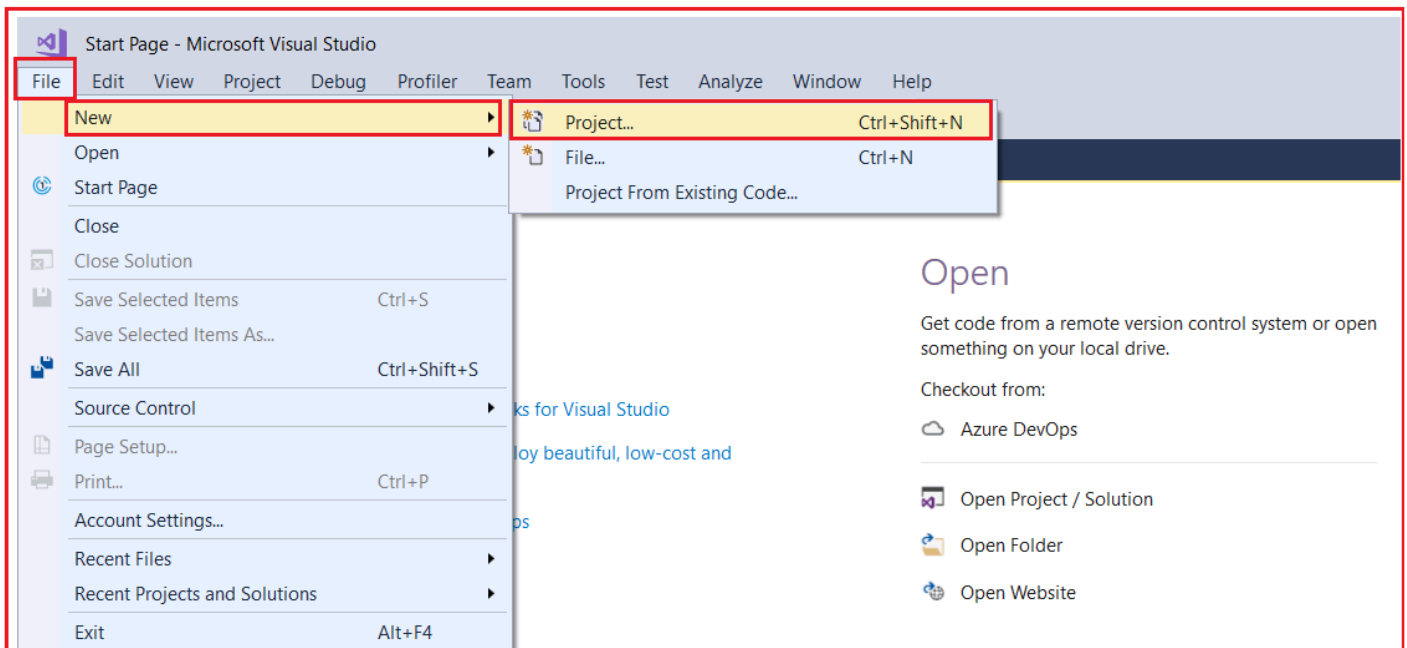
```

INSERT INTO Employee VALUES('Pam', 'Feale', 30000, 'IT')
INSERT INTO Employee VALUES('Smith', 'Male', 220000, 'Sales')
GO
-- Fetch the data from the Employee table
SELECT * FROM Employee
GO

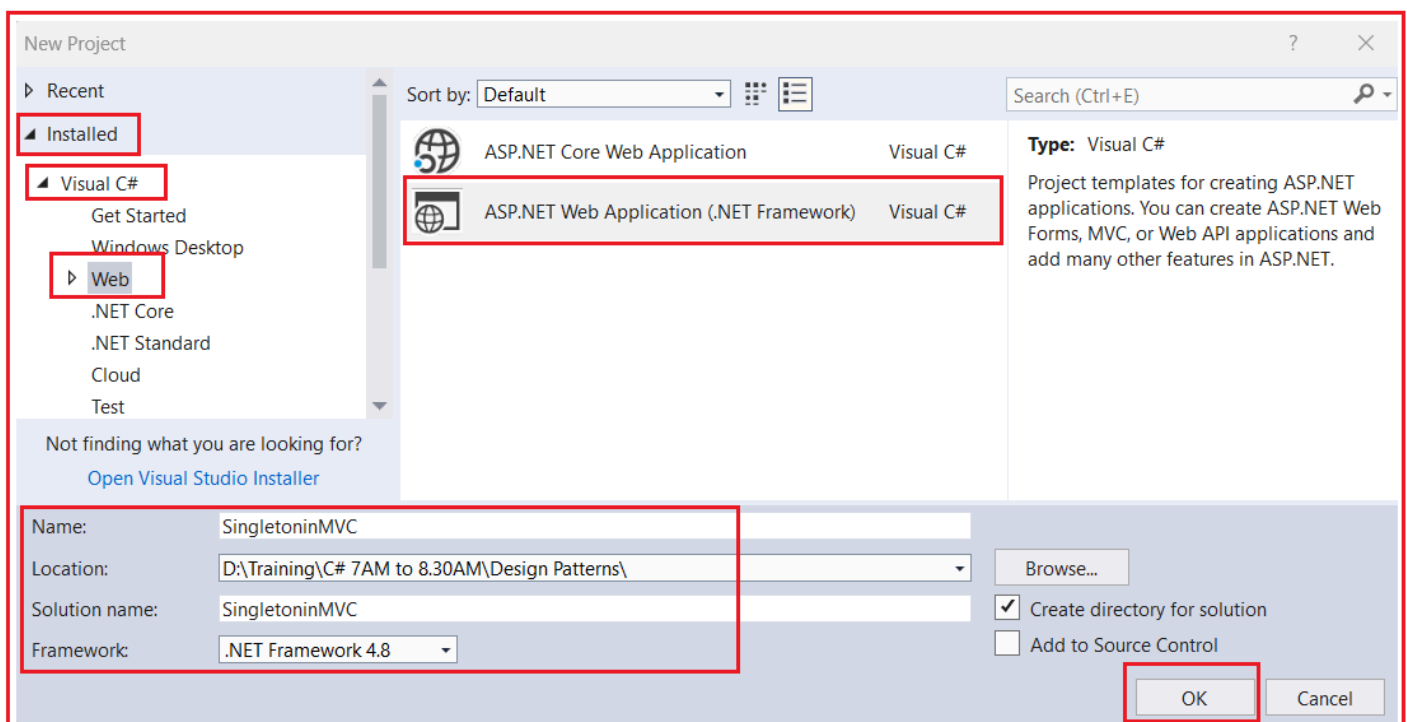
```

Step 2: Create a new ASP.NET MVC Application using Visual Studio

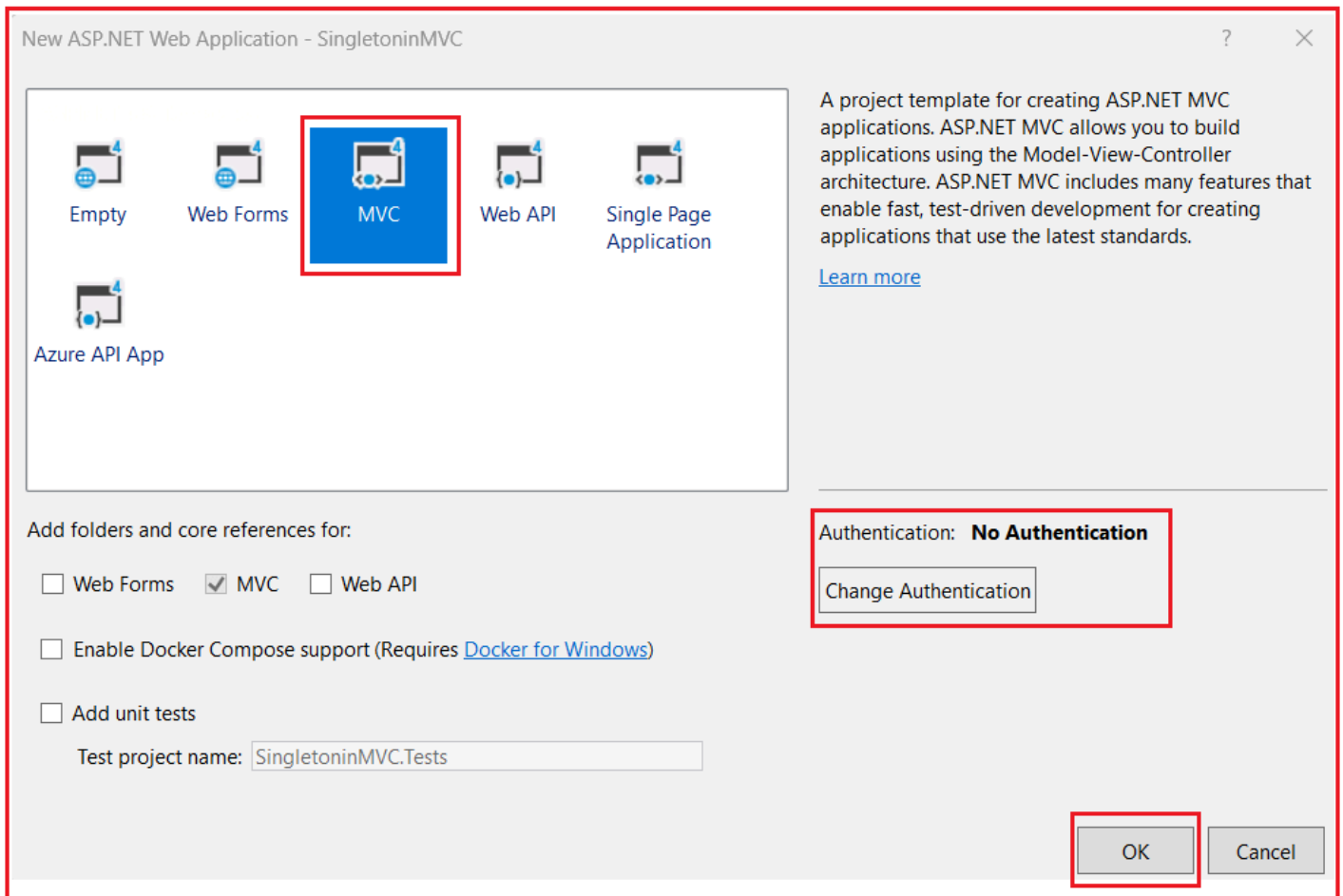
To Create a new ASP.NET MVC Application, open Visual Studio and then select **File => New => Project** from the context menu, as shown in the image below.



Once you click on the **File => New => Project**, the following window will open for selecting the New Project. Now, from the **New Project** window, from the left-hand side, select the **Web** option, which is inside the **Visual C#** under the **Installed** section. From the middle pane, select the **ASP.NET Web Application (.NET Framework)**, name the project **SingletoninMVC** and select the location where you want to create or save the project, select the .NET Framework version, and then click on the **OK** button as shown in the below image.



Once you click the **OK** button, the following **New ASP.NET Web Application** window will open for selecting the project Template. From this window, select the **MVC** project template, and then we are going to choose the Authentication type and. To do that, just click on the **Change Authentication** button, and a new dialog will pop up with the name **Change Authentication**. We are going to choose **No Authentication** and then click on the **OK** button, as shown in the below image.

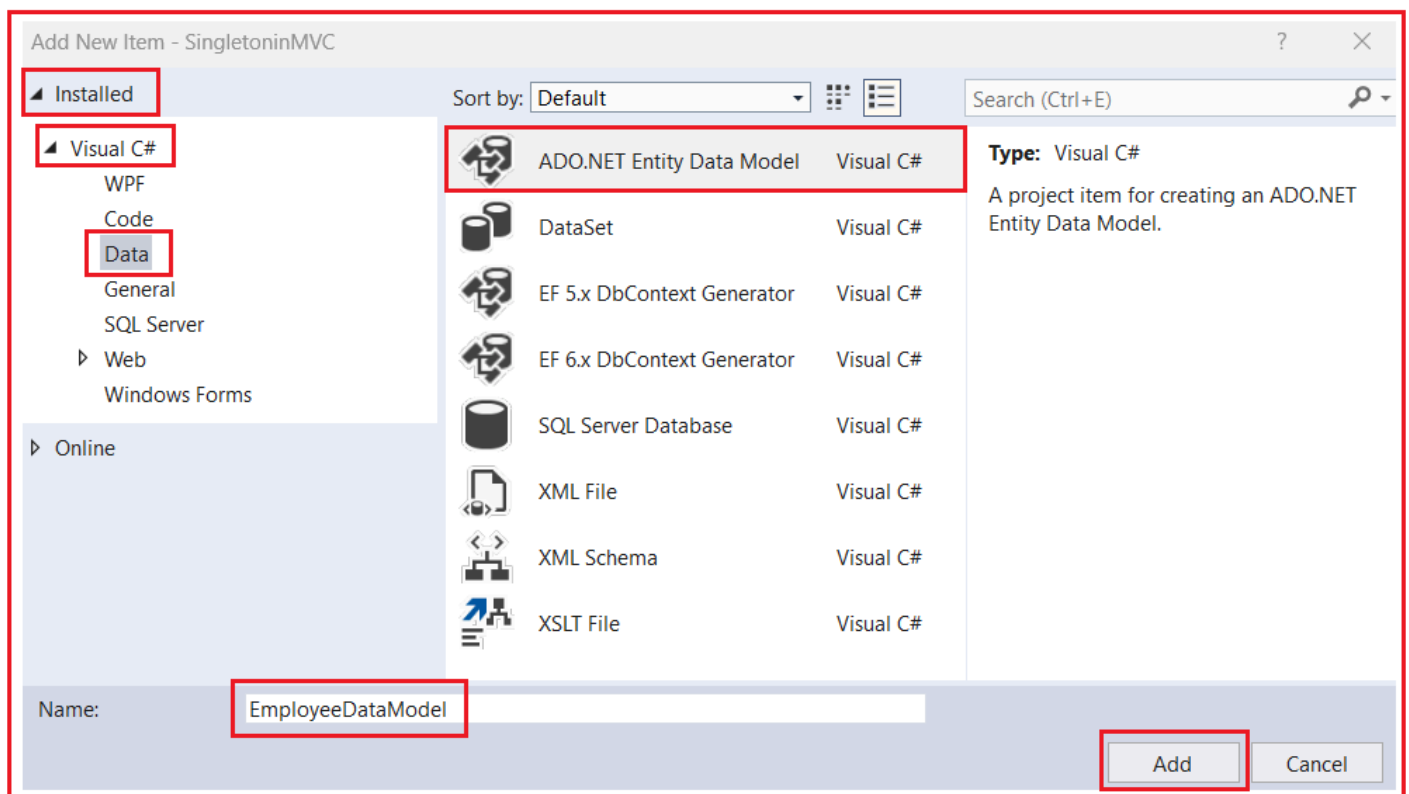


Once you click on the **OK** button, creating the project for us will take some time. So, we have created our ASP.NET MVC Application using Visual Studio.

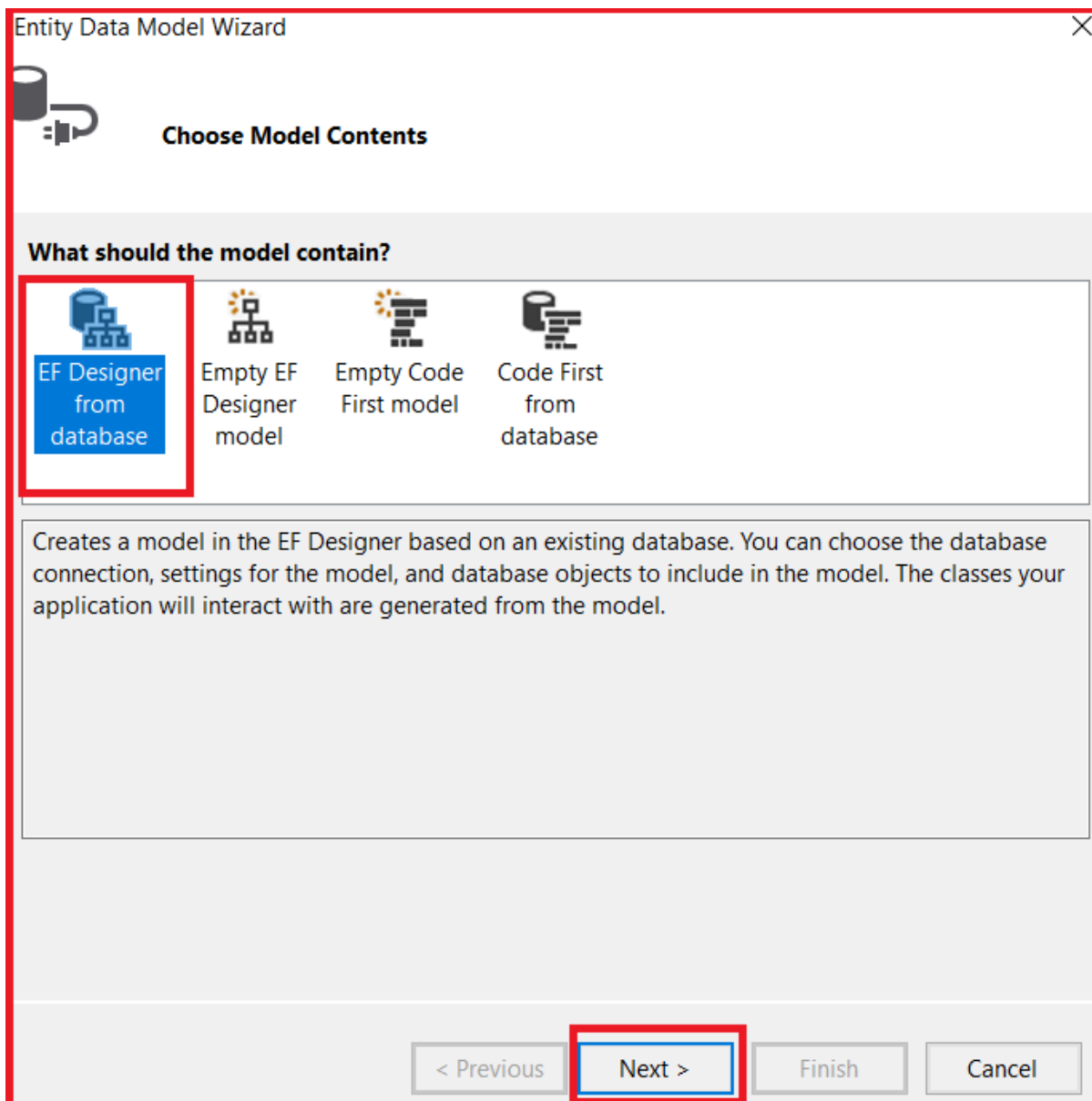
Step 3: Adding ADO.NET Entity Data Model

In the next step, we will use the Entity Framework Database First Approach to Communicate with the EmployeeDB and Employee we created in Step 1 from our ASP.NET MVC Application.

Add the ADO.NET Entity Data Model inside the Models Folder to do so. So, right-click on the Models folder, then select **Add => New Item** from the context menu to open the following Entity Data Model Wizard. From this wizard, from the left pane, select **Data**, which is inside **Visual C#**, which is under the **Installed** section. From the middle pane, select **ADO.NET Entity Data Model**, provide a meaningful name such as **EmployeeDataModel**, and then click on the **ADD** button as shown in the image below.



Once you click on the **Add** button, it will open a new window for selecting the Model type. Select the **Generate From Database** option from this window, as we will work with Entity Framework Database First Approach, and then click on the **Next** button, as shown in the image below.



Once you click on the **Next** button, it will open the **Choose Your Data Connection** Wizard. From this wizard, click the New Connection button to open the **Connection Properties** window. From this connection properties window, provide the necessary details (Database credentials, Sever Name, Data Provider), select the database (EmployeeDB), and then click on the **OK** button as shown in the image below.

Connection Properties

Enter information to connect to the selected data source or click "Change" to choose a different data source and/or provider.

Data source:
Microsoft SQL Server (SqlClient) Change...

Server name:
LAPTOP-2HN3PT8T\SQLEXPRESS Refresh

Log on to the server

Authentication: Windows Authentication

User name:

Password:

☐ Save my password

Connect to a database

☒ Select or enter a database name:
EmployeeDB

☐ Attach a database file:
 Browse...
Logical name:

Advanced...

Test Connection OK Cancel

Once you click on the **OK** button, it will take you back to **Choose Your Data Connection** Wizard. Here, in this window, you need to provide a meaningful name, such as **EmployeeDBContext**, for the Context class, and that name will also be the **Connection String** name that will be created in the **Web.config** file. Finally, click on the **Next** button as shown below.

Entity Data Model Wizard

 **Choose Your Data Connection**

Which data connection should your application use to connect to the database?

laptop-2hn3pt8t\sqlexpress.EmployeeDB.dbo New Connection...

This connection string appears to contain sensitive data (for example, a password) that is required to connect to the database. Storing sensitive data in the connection string can be a security risk. Do you want to include this sensitive data in the connection string?

☐ No, exclude sensitive data from the connection string. I will set it in my application code.

☐ Yes, include the sensitive data in the connection string.

Connection string:

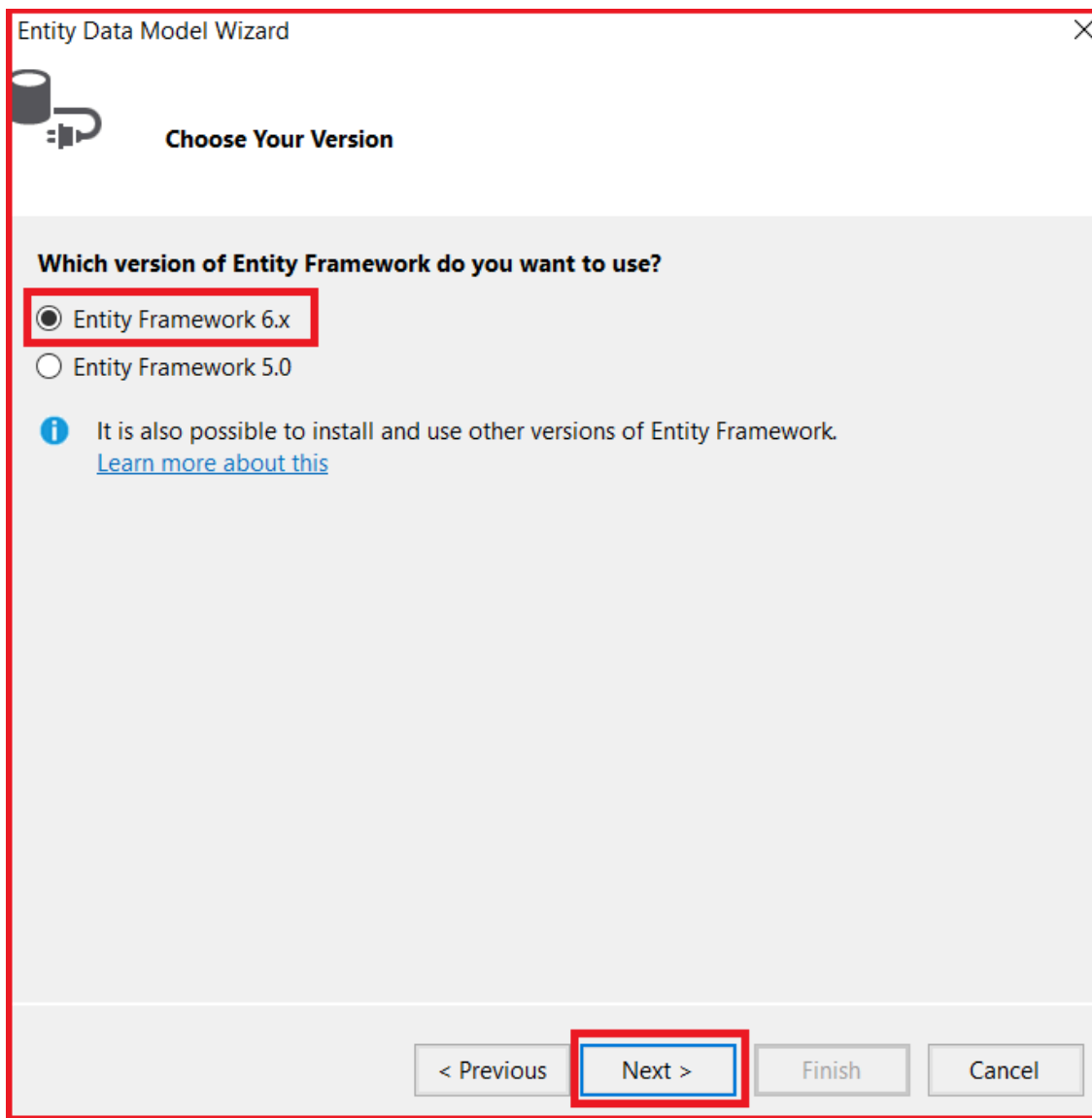
```
metadata=res://*/Models.EmployeeDataModel.csdl|res://*/Models.EmployeeDataModel.ssdl|
res://*/Models.EmployeeDataModel.msl;provider=System.Data.SqlClient;provider connection
string="data source=LAPTOP-2HN3PT8T\SQLEXPRESS;initial catalog=EmployeeDB;integrated
security=True;MultipleActiveResultSets=True;App=EntityFramework"
```

☒ Save connection settings in Web.Config as:

EmployeeDBContext

< Previous **Next >** Finish Cancel

Once you click the **Next** button, it will open the **Choose Your Version** wizard for selecting the Entity Framework version. From this window, select **Entity Framework 6.x** and click on the **Next** button, as shown in the below image. Sometimes, this window will not open if the latest Entity Framework version is installed.



Once you select the **Entity Framework Version** and click the **Next** button, it will open the **Choose Your Database Objects and settings** wizard. From this wizard, select the **Employee** object, provide a meaningful namespace name, and finally click on the **Finish** button, as shown in the image below.

Entity Data Model Wizard

Choose Your Database Objects and Settings

Which database objects do you want to include in your model?

- ☒ Tables
 - ☒ dbo
 - ☒ Employee
- ☐ Views
- ☐ Stored Procedures and Functions

☒ Pluralize or singularize generated object names

☒ Include foreign key columns in the model

☐ Import selected stored procedures and functions into the entity model

Model Namespace:

Models

< Previous Next > **Finish** Cancel

That's it. We have added the Entity Data Model from our ASP.NET MVC Application.

Step 4: Creating the Singleton Class for Logging Functionality

Now, we need to create the Singleton Class to provide the Exception Logging Functionality. So, add a folder with the name `Logger` at the root directory of our MVC Application.

`ILog.cs`

Once you add the `Logger` Folder, add a class file with the name `ILog.cs` and copy and paste the following code into it. This is an interface, and this interface provides one method, i.e., `LogException`. The child class is going to provide the implementation for this `LogException` method. In our example, the Singleton class will implement this interface and provide implementations for the `LogException` method.

```
namespace SingletoninMVC.Logger
{
    public interface ILog
    {
        void LogException(string message);
    }
}
```

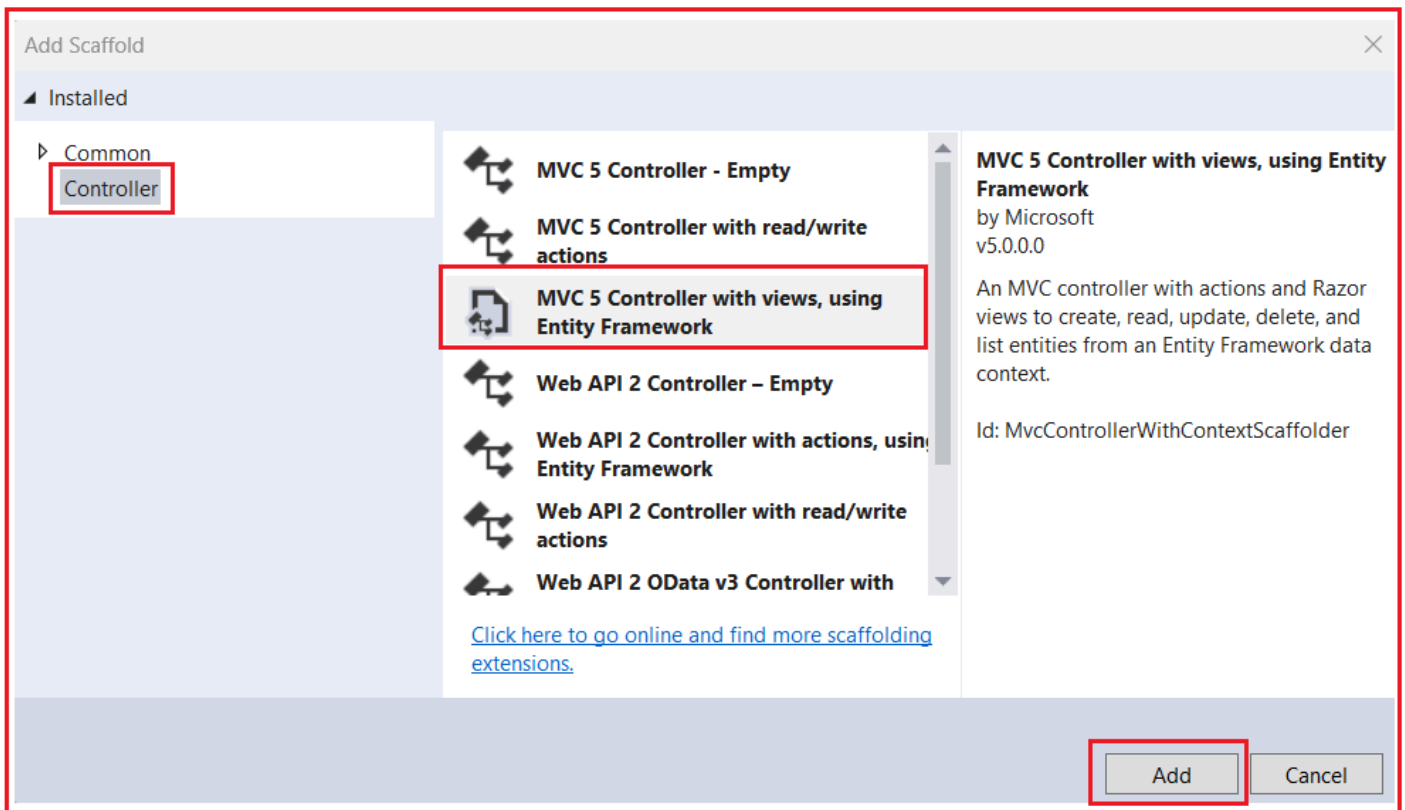
`Log.cs`

Now, we need to create the Singleton Class by implementing the ILog interface. So, create a class file with the name **Log.cs** within the Logger folder and then copy and paste the following code into it. This Log class is nothing but the Singleton class, which uses Eager loading, which is thread-safe in a multithread environment. Here, we are following the Rules of the Singleton Design Pattern by making the class Sealed (to restrict inheritance), creating the private parameterless constructor (to restrict class instantiation from outside the class), creating a public method to access the only instance of the Log class, i.e. (GetInstance method). Once the client gets the Log instance, then the client using the Log instance can call the LogException method by passing the exception message as an argument.

```
using System;
using System.IO;
using System.Text;
namespace SingletoninMVC.Logger
{
    public sealed class Log : ILog
    {
        //Private Constructor to Restrict Class Instantiation from outside
        the Log class
        private Log()
        {
        }
        //Creating Log Instance using Eager Loading
        private static readonly Log LogInstance = new Log();
        //Returning the Singleton LogInstance
        //This Method is Thread Safe as it uses Eager Loading
        public static Log GetInstance()
        {
            return LogInstance;
        }
        //This Method Log the Exception Details in a Log File
        public void LogException(string message)
        {
            //Create the Dynamic File Name
            string fileName =
                $"Exception_{DateTime.Now.ToShortDateString()}.log";
            //Create the Path where you want to Create the Log file
            string logFilePath = $"
                {AppDomain.CurrentDomain.BaseDirectory}\\{fileName}";
            //Build the String Object using StringBuilder for a Better
            Performance
            StringBuilder sb = new StringBuilder();
            sb.AppendLine("-----");
            sb.AppendLine(DateTime.Now.ToString());
            sb.AppendLine(message);
            //Write the StringBuilder Message into the Log File Path using
            StreamWriter Object
            using (StreamWriter writer = new StreamWriter(logFilePath,
                true))
            {
                writer.Write(sb.ToString());
                writer.Flush();
            }
        }
    }
}
```

Step 5: Adding Employee Controller:

Once the Log Singleton class is ready, we need to use the LogException method in our MVC Application. So, add one controller with the name Employee. Right-click on the Controllers folder and then select Add=> Controller from the context menu, which will open the Add Scaffold window. Here, you need to choose the **MVC 5 Controller with views using the Entity Framework** template, as shown in the image below.



Once you click on the Add button, it will open the following Add Controller window. Here, we need to provide the Controller information. From this window, provide the Model class as Employee (SingletoninMVC.Models), Data Context class as EmployeeDbContext (SingletoninMVC.Models), and Controller name as EmployeeController, select all the checkboxes under the Views section and finally, click on the **Add** button as shown in the below image.

Add Controller

Model class: Employee (SingletoninMVC.Models)

Data context class: EmployeeDBContext (SingletoninMVC.Models) +

☐ Use async controller actions

Views:

☒ Generate views

☒ Reference script libraries

☒ Use a layout page:

(Leave empty if it is set in a Razor _viewstart file)

Controller name: EmployeesController

Add Cancel

Once you click the Add button, it will create an Employee controller with the required action methods and Views, which facilitates the CRUD Operations on the employee model. If you are getting errors, build the application and follow the same steps to create the controller.

Step 6: Using Exception Logging inside the EmployeeController

To use the Singleton Log class LogException method to the Log the Exception detail, please modify the Employee Controller class as follows.

```
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.Entity;
using System.Linq;
using System.Net;
using System.Web;
using System.Web.Mvc;
using SingletoninMVC.Logger;
using SingletoninMVC.Models;
namespace SingletoninMVC.Controllers
{
    public class EmployeeController : Controller
    {
        private ILog _ILog;
        private EmployeeDBContext db = new EmployeeDBContext();
        public EmployeeController()
        {
            //Get the Singleton Log Instance
            _ILog = Log.GetInstance();
        }
    }
}
```

```

    }
    //Whenever Any Exception Occurred, the following OnException Method
will Execute
    protected override void OnException(ExceptionContext filterContext)
    {
        //First, Log the Exception Details
        _ILog.LogException(filterContext.Exception.ToString());
        //Then set that the Exception is Handled
        filterContext.ExceptionHandled = true;
        //Then Redirect to the Error view
        this.View("Error").ExecuteResult(this.ControllerContext);
    }
    // GET: Employee
    public ActionResult Index()
    {
        return View(db.Employees.ToList());
    }
    // GET: Employee/Details/5
    public ActionResult Details(int? id)
    {
        if (id == null)
        {
            return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
        }
        Employee employee = db.Employees.Find(id);
        if (employee == null)
        {
            throw new Exception("Employee Not Found");
        }
        return View(employee);
    }
    // GET: Employee/Create
    public ActionResult Create()
    {
        return View();
    }
    // POST: Employee/Create
    // To protect from overposting attacks, please enable the specific
properties you want to bind to, for
    // more details see https://go.microsoft.com/fwlink/?LinkId=317598.
    [HttpPost]
    [ValidateAntiForgeryToken]
    public ActionResult Create([Bind(Include =
"Id,Name,Gender,Salary,Department")] Employee employee)
    {
        if (ModelState.IsValid)
        {
            db.Employees.Add(employee);
            db.SaveChanges();
            return RedirectToAction("Index");
        }
        return View(employee);
    }
    // GET: Employee/Edit/5

```

```

public ActionResult Edit(int? id)
{
    if (id == null)
    {
        return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    }
    Employee employee = db.Employees.Find(id);
    if (employee == null)
    {
        return HttpNotFound();
    }
    return View(employee);
}
// POST: Employee/Edit/5
// To protect from overposting attacks, please enable the specific
properties you want to bind to, for
// more details see https://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public ActionResult Edit([Bind(Include =
"Id,Name,Gender,Salary,Department")] Employee employee)
{
    if (ModelState.IsValid)
    {
        db.Entry(employee).State = EntityState.Modified;
        db.SaveChanges();
        return RedirectToAction("Index");
    }
    return View(employee);
}
// GET: Employee/Delete/5
public ActionResult Delete(int? id)
{
    if (id == null)
    {
        return new HttpStatusCodeResult(HttpStatusCode.BadRequest);
    }
    Employee employee = db.Employees.Find(id);
    if (employee == null)
    {
        return HttpNotFound();
    }
    return View(employee);
}
// POST: Employee/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public ActionResult DeleteConfirmed(int id)
{
    Employee employee = db.Employees.Find(id);
    db.Employees.Remove(employee);
    db.SaveChanges();
    return RedirectToAction("Index");
}

```

```

        protected override void Dispose(bool disposing)
        {
            if (disposing)
            {
                db.Dispose();
            }
            base.Dispose(disposing);
        }
    }
}

```

The Home Controller Index action method is configured as the landing page by default. Let us modify the RouteConfig.cs class file, which you can find within the App_Start folder as follows. Now, we are making our Employee Controller and Index action method the default route.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Web.Routing;
namespace SingletoninMVC
{
    public class RouteConfig
    {
        public static void RegisterRoutes(RouteCollection routes)
        {
            routes.IgnoreRoute("{resource}.axd/{*pathInfo}");
            routes.MapRoute(
                name: "Default",
                url: "{controller}/{action}/{id}",
                defaults: new { controller = "Employee", action = "Index",
id = UrlParameter.Optional }
            );
        }
    }
}

```

That's it. We have done with our implementation.

Testing the Application.

Now run the application, and try to do something which should give some exceptions. Then, you will see that all the exceptions will be logged under the file created by the logger class. You can find the Exception Log file as shown in the below path.

> This PC > DATA (D:) > Training > C# 7AM to 8.30AM > Design Patterns > SingletoninMVC > SingletoninMVC				
Name	Date modified	Type	Size	
Content	21-03-2023 20:10	File folder		
Controllers	21-03-2023 21:12	File folder		
fonts	21-03-2023 20:10	File folder		
Logger	21-03-2023 20:42	File folder		
Models	21-03-2023 20:22	File folder		
obj	21-03-2023 20:10	File folder		
Properties	21-03-2023 20:10	File folder		
Scripts	21-03-2023 20:10	File folder		
Views	21-03-2023 20:58	File folder		
Exception_21-03-2023	21-03-2023 21:13	Text Document	3 KB	
favicon	21-03-2023 20:10	ICO File	32 KB	
Global.asax	21-03-2023 20:10	ASAX File	1 KB	
Global.asax.cs	21-03-2023 20:10	Visual C# Source F...	1 KB	

This Proves that a singleton design pattern comes in handy when we need to have a single instance of the object. In the next article, I am going to discuss [How to Implement Singleton Design Pattern Real-Time Example Caching using C#](#). In this article, I explain the Singleton Design Pattern in a Real-Time Example of Exception Logging in C# using an ASP.NET MVC Application.